

Jetson Camera Detection Service

Installation and Setup Guide

This guide explains how to set up the camera detection service on a Jetson device. No programming experience is required. Follow the steps in order and run the commands exactly as shown.

Overview

The Jetson is a compact computer made by NVIDIA. Once configured with this service, it performs three functions:

1. **Connects to your cameras** over the local network.
2. **Analyses the video using AI**, running an object detection model (for example, detecting people or vehicles) on the Jetson's onboard graphics processor.
3. **Reports its findings** so that other software can request the current detections for any camera at any time.

Setup is performed once. After that, the device retains its camera configuration and starts the service automatically each time it is powered on.

Estimated time: approximately 45 to 75 minutes. The majority of this is an unattended one time download, which does not require supervision.

Step 1: Identify your device

Two Jetson models are supported, and each requires a slightly different setup. You do

not need prior knowledge of either; you only need to determine which one you have.

Power on the device, open the **Terminal** application (the window in which commands are entered), type the following, and press **Enter**:

```
cat /proc/device-tree/model; echo
```

Interpret the result as follows:

If the output contains	Device	Instructions to follow
Orin Nano	The newer model	Continue with the steps below
Nano (without Orin)	The older model	Proceed to Appendix A

Tip: Copy and paste the commands rather than typing them. In the Jetson terminal, the paste shortcut is usually **Ctrl + Shift + V**.

Step 2: Run the setup script

A setup script is provided that performs the complete installation, configures the service, and prepares it to run. This is the recommended method for all users.

First, change into the project folder:

```
cd ~/tensort
```

Then run the script that corresponds to your device.

For the newer Orin Nano:

```
chmod +x setup_orin.sh
./setup_orin.sh
```

For the older Nano (please review Appendix A first for one important note):

```
chmod +x setup_nano_py36.sh
./setup_nano_py36.sh
```

The script reports its progress as it runs and can be left unattended. If it stops with a red **ERROR** message, consult the **Troubleshooting** section, which lists the common causes and their resolutions.

The script is safe to re run. If it is interrupted (for example, by a power loss or a closed window), simply run the same command again. It resumes from where it stopped and does not disrupt any work already completed.

What the script performs

For transparency, the script carries out the following:

1. **Downloads the NVIDIA software stack**, comprising the AI and video processing components the Jetson requires. This is the large, time consuming download.
2. **Enables maximum performance**, allowing the device to operate at full capacity for multiple cameras.
3. **Creates an isolated environment** for the application and its dependencies.
4. **Builds the AI model for this specific device.** The model file must be compiled on the device that will run it; a copy taken from another machine will not load.
5. **Generates the configuration file** so the service is ready to start.

The following optional settings are available, although most users will not require them:

Setting	Effect
<code>SKIP_APT=1</code>	Skips the large download (if already completed)
<code>SKIP_ENGINE=1</code>	Skips rebuilding the AI model
<code>MODEL=yolov8n</code>	Selects which AI model to use

For example, `SKIP_APT=1 ./setup_orin.sh` runs the setup but omits the large download.

Step 3: Start the service and confirm it is running

When the script has finished, start the service:

```
cd ~/tensort
source .venv_trt/bin/activate
python3 main.py
```

The service starts and reports that it is listening on **port 8080**. Keep this window open; closing it stops the service. Step 5 explains how to run the service automatically.

Confirming the service is operational: from any computer on the same network, query the device's health endpoint:

```
curl http://<jetson-ip>:8080/health
```

Replace `<jetson-ip>` with the device's network address. A response containing `"pipeline_ready": true` confirms that the service is functioning correctly.

Step 4: Add your cameras

Cameras are registered by sending the service a short request. Each camera requires two pieces of information:

- **The video source address** (`source_url`), which specifies where the stream is located.
- **A name of your choosing** (`camera_uuid`), such as `front-door`.

The service supports a range of camera formats, not only RTSP. Provide whichever address your camera offers:

Camera format	Example address
RTSP (most IP cameras)	<code>rtsp://user:pass@192.168.1.50:554/stream1</code>
WebRTC / WHEP	<code>whep://host/stream/whep</code>
HLS or MJPEG (web video)	<code>http://host/stream/index.m3u8</code>
RTMP	<code>rtmp://host/app/streamkey</code>
SRT	<code>srt://host:8890?streamid=...</code>

To add a camera, run the following, substituting your camera's address and chosen name:

```
curl -X POST http://<jetson-ip>:8080/cameras \
  -H "Content-Type: application/json" \
  -d '{
    "source_url": "rtsp://user:pass@192.168.1.50:554/stream1",
    "camera_uuid": "front-door"
  }'
```

A successful response indicates that the camera has been added, saved, and is now being analysed. The same command works for any camera format; only the `source_url` changes.

Additional commands:

```
# List all registered cameras
curl http://<jetson-ip>:8080/cameras

# View the current detections for one camera
curl http://<jetson-ip>:8080/cameras/front-door/latest

# Save an image from a camera, with detections drawn on it
curl http://<jetson-ip>:8080/cameras/front-door/snapshot.jpg --output
snap.jpg

# Remove a camera
curl -X DELETE http://<jetson-ip>:8080/cameras/front-door
```

Each camera is added only once. Registrations are stored on the device and are restored automatically whenever it restarts.

Step 5: Configure automatic startup (recommended)

To avoid starting the service manually, you can configure the device to launch it automatically at power on. Create a startup file:

```
sudo nano /etc/systemd/system/jetson-cameras.service
```

Paste the text below, replacing `YOUR_USERNAME` with your actual login name in all three locations:

```
[Unit]
```

```
Description=Jetson Camera Service
After=network-online.target
Wants=network-online.target

[Service]
Type=simple
User=YOUR_USERNAME
WorkingDirectory=/home/YOUR_USERNAME/tensort
Environment=PATH=/usr/local/cuda/bin:/usr/bin:/bin
Environment=LD_LIBRARY_PATH=/usr/local/cuda/lib64
ExecStartPre=+/usr/sbin/nvpmode1 -m 0
ExecStartPre=+/usr/bin/jetson_clocks
ExecStart=/home/YOUR_USERNAME/tensort/.venv_trt/bin/python3 main.py
Restart=on-failure
RestartSec=5

[Install]
WantedBy=multi-user.target
```

Save and close the file (in the `nano` editor, press **Ctrl + O**, then **Enter**, then **Ctrl + X**).

Then enable and start the service:

```
sudo systemctl daemon-reload
sudo systemctl enable jetson-cameras
sudo systemctl start jetson-cameras
```

To view the service running in real time, use `journalctl -u jetson-cameras -f`. Press **Ctrl + C** to stop viewing the log; this does not stop the service itself.

Troubleshooting

Locate your symptom in the left column and apply the corresponding resolution. An understanding of the underlying cause is not required.

Symptom	Resolution
The large NVIDIA download did not complete	Run the setup script again. It resumes safely.
Camera added, but no detections or a blank image	The camera address is incorrect or unreachable. Verify the address, the username and password, and the device's network access to the camera.
The health check reports <code>"pipeline_ready": false</code>	The AI model failed to load. Re run the setup script so it rebuilds the model on this device.
An error reports the AI model or engine is not found	The model was not built on this device. Re run the setup script, which builds it automatically.
Video appears slow or delayed	Confirm that maximum performance is enabled (the script does this) and that the number of simultaneous cameras is reasonable.
You wish to start over completely	Stop the service and delete the file <code>jetson_cameras.db</code> in the <code>tensorrt</code> folder. The camera list is cleared and the device starts fresh.

When in doubt, re run the setup script. It is designed to be run repeatedly and resolves most setup issues automatically.

Appendix A: The older Jetson Nano

The main guide is written for the newer Orin Nano. If Step 1 indicated that you have the older Nano, use the script provided for that model:


```
cd ~/tensort
chmod +x setup_nano_py36.sh
./setup_nano_py36.sh
```

Important note for the older Nano. The AI detection component is written for the newer model's software. On the older Nano, the setup script installs the complete environment and prepares it, but the detection step requires a minor code adjustment by a developer before it produces results. The camera handling functions operate without modification; only the detection step needs this adjustment. The script displays a reminder to this effect when it finishes.

All other procedures, including adding cameras, checking service health, and configuring automatic startup, are identical to the main guide.

Appendix B: Manual installation (for advanced users)

The setup script in Step 2 runs the commands below on your behalf. They are provided here for those who prefer to run each step manually or who wish to understand the process in detail. If you used the script, this section can be disregarded.

```
# 1. Download the NVIDIA software stack and build tools
sudo apt-get update
sudo apt-get install -y nvidia-jetpack
sudo apt-get install -y python3-venv python3-dev build-essential \
    gstreamer1.0-plugins-good gstreamer1.0-plugins-bad gstreamer1.0-libav

# 2. Enable maximum performance
sudo nvmodel -m 0 && sudo jetson_clocks

# 3. Create the environment
cd ~/tensort
python3 -m venv --system-site-packages .venv_trt
source .venv_trt/bin/activate
```

```
python3 -m pip install --upgrade pip setuptools wheel
export PATH=/usr/local/cuda/bin:$PATH CUDA_ROOT=/usr/local/cuda
export LD_LIBRARY_PATH=/usr/local/cuda/lib64:$LD_LIBRARY_PATH
pip install -r requirements-jetson.txt

# 4. Build the AI model on this device
/usr/src/tensorrt/bin/trtexec --onnx=models/yolov8n.onnx \
    --saveEngine=models/yolov8n.engine --fp16 --
memPoolSize=workspace:2048

# 5. Create the configuration and run the service
cp .env.example .env          # then set
DET_ENGINE=./models/yolov8n.engine
python3 main.py               # the service starts on port 8080
```

Note for the older Nano: use `requirements.txt` instead of `requirements-jetson.txt`, install `pycuda==2020.1`, and build the model with `--workspace=1024` in place of `--memPoolSize=workspace:2048`. The `setup_nano_py36.sh` script handles all of these differences automatically.

Installation and setup guide for the Jetson camera detection service. The newer Orin Nano is covered in the main guide; the older Nano is covered in Appendix A.